Shader Code Examples

Sunday, April 19, 2026 10:04 PM

1. Vertex Shader

This version has a normal vector for lighting calculations and texture coordinates for the surface.

```
#version 330 core // OpenGL version 3.3. Core profile

layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aNormal; // For lighting
layout (location = 2) in vec2 aTexCoord; // For textures

out vec3 Normal;
out vec3 FragPos;
out vec2 TexCoord;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main() {
   // Standard 3D transformation
   gl_Position = projection * view * model * vec4(aPos, 1.0);

   // Pass world-space position and normals to Fragment Shader for lighting
   FragPos = vec3(model * vec4(aPos, 1.0));
   Normal = mat3(transpose(inverse(model))) * aNormal;

   TexCoord = aTexCoord;
}
```

2. Fragment Shader (with AA & Compositing)

This shader samples a texture, applies a basic Blinn-Phong light model, and demonstrates how a post-processing pass would implement FXAA-style smoothing and UI Compositing.

```
#version 330 core  // OpenGL version 3.3. Core profile


out vec4 FragColor;

in vec3 Normal;
in vec3 FragPos;
in vec2 TexCoord;

uniform sampler2D mainTexture;   // The 3D object's texture
```

```
uniform sampler2D uiOverlay;     // A 2D UI texture for compositing
uniform vec3 lightPos;
uniform bool enableFXAA;         // Toggle for simple AA enhancement

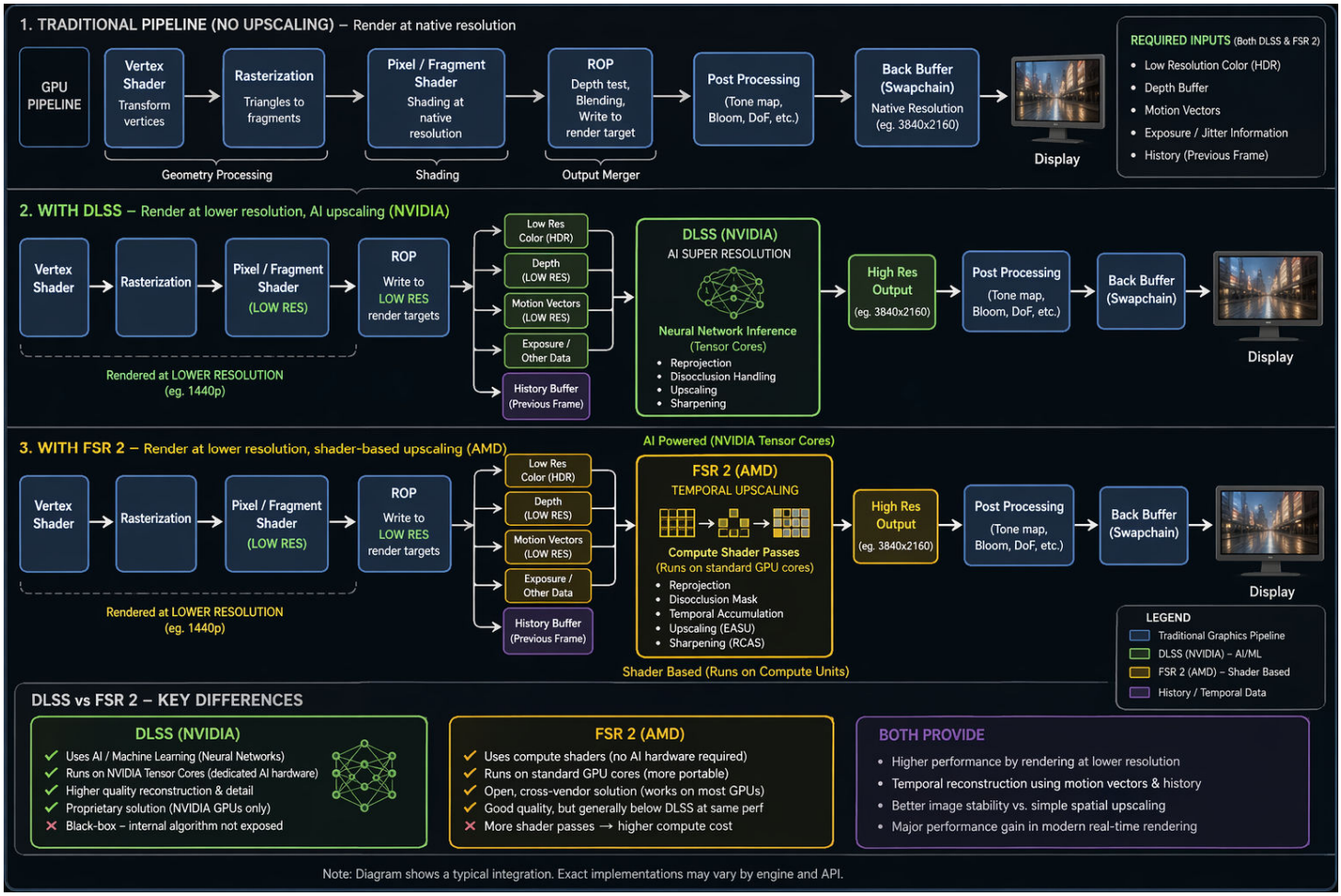
void main() {
   // 1. TEXTURE SAMPLING
   vec4 texColor = texture(mainTexture, TexCoord);

   // 2. BASIC LIGHTING (Enhancement)
   vec3 norm = normalize(Normal);
   vec3 lightDir = normalize(lightPos - FragPos);
   float diff = max(dot(norm, lightDir), 0.0);
   vec3 lighting = diff * vec3(1.0, 1.0, 1.0); // White light

   vec4 result = vec4(lighting * texColor.rgb, texColor.a); //RGBA, a -- alpha, transparency or opacity

   // 3. DISPLAY ENHANCEMENT: Simple Post-AA (Edge Blur)
   // In a real FXAA pass, you would sample neighbors to detect edges.
   if(enableFXAA) {
      vec2 offset = vec2(1.0 / 800.0, 1.0 / 600.0); // Screen resolution offset
      vec4 neighbor = texture(mainTexture, TexCoord + offset);
      result = mix(result, neighbor, 0.5); // Simple blur to soften edges
   }

   // 4. COMPOSITING (Overlaying UI)
   vec4 uiColor = texture(uiOverlay, TexCoord);
   // Composite UI on top of the 3D scene using alpha blending
   FragColor = mix(result, uiColor, uiColor.a); //a built-in GLSL tool used for Linear Interpolation
   //May be translated into an EXPORT_COLOR instruction, which has 4 source register operands.
//EXPORT_COLOR is not always shown literally in public SASS docs, but the hardware has a pixel
//export stage
//The export writes into a fixed-function queue, not memory
//After this, the warp is done—ROP takes over. ROP: Raster Operations Pipeline (sometimes called
Render Output Unit)
}
```



FSR4 starts to be similar to DLSS, i.e., using specialized HW for SS.

```
struct RayPayload {
   bool hit;
   float t;
};

bool trace_shadow_ray(float3 origin, float3 dir)
{
   RayPayload payload;
   payload.hit = false;
   payload.t = 0.0;

   RayDesc ray;
   ray.Origin = origin;
   ray.Direction = normalize(dir);
   ray.TMin = 0.001;
   ray.TMax = 10000.0;

   // Intersects acceleration structure (BVH)
   trace_ray(sceneBVH, ray, payload);

   return payload.hit;
}

float4 fragment_main(FragmentInput input) : SV_Target
```

```
{
    float3 worldPos = input.worldPos;
    float3 normal   = normalize(input.normal);

    float3 lightPos = uLight.position;
    float3 lightColor = uLight.color;

    // --- Lighting direction ---
    float3 L = lightPos - worldPos;
    float distToLight = length(L);
    L = normalize(L);

    // --- Shadow ray ---
    float3 shadowOrigin = worldPos + normal * 0.001; // avoid acne
    bool inShadow = trace_shadow_ray(shadowOrigin, L);

    // --- Diffuse lighting ---
    float NdotL = max(dot(normal, L), 0.0);
    float3 diffuse = NdotL * lightColor;

    // --- Shadow factor ---
    float shadow = inShadow ? 0.0 : 1.0;

    float3 finalColor = diffuse * shadow;

    return float4(finalColor, 1.0);
}
```